

Planet Pulse

See what Earth is signaling now.

全球环境态势感知平台
项目整体逻辑与架构文档

作者: John Zhou

日期: 2026年7月

许可证: Apache License 2.0

GitHub: github.com/jzjzzzzzz/PlanetPulse

在线演示: planet-pulse-eta.vercel.app

目录

1. 项目概述与产品定位
2. 技术架构总览
3. 数据流与API设计
4. 核心算法：热点评分系统
5. 核心算法：个人相关性评分
6. 核心算法：Haversine距离计算
7. 组件树与UI架构
8. 3D地球渲染引擎
9. 数据源与容错机制
10. 部署与运维
11. 测试体系
12. 隐私与数据伦理
13. 未来规划

1 项目概述与产品定位

1.1 项目简介

Planet

Pulse

是一个全球环境态势感知网站。它将分散的环境数据转化为清晰的全球概况和个性化的本地环境信号。平台结合了电影级交互式3D地球、NASA实时环境事件数据、可解释的热点排名系统以及基于位置感知的个人相关性信息。

1.2 产品定位

- 类型：环境态势感知网站（非紧急预警服务）
- 功能：全球环境监测仪表盘 + 地球系统教育工具 + NASA开放数据消费者级界面
- 明确不是：紧急警报或预警系统、NASA官方产品、本地应急服务替代品

1.3 核心功能

功能模块	说明
3D地球	交互式暗色主题地球，大气光晕，彩色事件标记，动画脉冲环
热点排名	0-100确定评分，8大因子，每项评分附带人类可读解释
个人相关性	距离 x 严重性 x 时效性 x 时间上下文，四维加权评分
实时数据	NASA EONET v3 (环境事件) + FIRMS (火灾检测)
响应式设计	桌面全布局 + 移动端简化布局，375px至大屏全适配

1.4 技术栈

层级	技术
框架	Next.js 16 (App Router)
语言	TypeScript (strict mode)
样式	Tailwind CSS v4
3D引擎	react-globe.gl + Three.js
图标	Lucide React
数据校验	Zod
测试	Vitest
部署	Vercel
包管理	npm

2 技术架构总览

2.1 整体架构

Planet Pulse 采用 Next.js App Router 架构，遵循客户端-服务器分离原则。

Client（浏览器端）：

- 3D Globe (dynamic import, SSR disabled)
- Layer Panel (事件分类过滤)
- Hotspot Panel (全球热点排名)
- Local Signal Bar (个人相关性信号)
- Event Details (事件详情弹窗)
- Observation Timeline (观测历史时间线)
- Mobile Event Sheet (移动端底部抽屉)

Server (Next.js API Routes)：

- /api/events -> NASA EONET v3 API
- /api/fires -> NASA FIRMS API
- /api/location -> Vercel IP Geolocation
- Scoring Engine (纯函数，独立可测)

2.2 关键架构决策

决策	理由
Globe组件	使用 dynamic import + ssr:false，Three.js/WebGL仅客户端加载
数据获取	全部通过服务端 API Routes，API密钥不暴露给客户端
评分逻辑	纯函数隔离，零副作用，100%可单元测试
无状态设计	无数据库、无认证、无CMS - 完全无状态
容错机制	三级降级：Live -> Stale Cache -> Fallback Sample Data

2.3 项目文件结构

```
day03/
app/
  layout.tsx    # 根布局 (字体/元数据)
  page.tsx      # 主页面 (状态管理+组件编排)
  globals.css   # 全局样式 (CSS变量+暗色主题)
  api/
    events/route.ts # 事件API (EONET)
    fires/route.ts  # 火灾API (FIRMS)
    location/route.ts # 位置API (IP地理定位)
    health/route.ts # 健康检查
    satellite/route.ts # 卫星影像代理
  components/
    globe/        # 3D地球组件
    layout/       # 布局组件 (面板/状态栏)
```

```
events/      # 事件展示组件
ui/          # 通用UI组件
lib/
scoring/     # 评分算法 (核心逻辑)
nasa/        # NASA数据获取+规范化
geo/         # 地理计算
globe/       # 地球可视化工具
location/    # 位置解析
formatting/  # 日期/时间格式化
types/
environment.ts # 核心类型定义
tests/       # 单元测试
```

3 数据流与API设计

3.1 数据获取流程

数据流遵循 Server -> Client 单向模式，保证API密钥安全：

1. 用户访问页面 -> 浏览器加载React应用
2. Client fetch("/api/events") -> Next.js API Route
3. API Route fetch(EONET_API_URL) -> NASA EONET v3
4. EONET返回原始JSON -> Zod Schema验证
5. normalizeEvent() 规范化 -> EnvironmentalEvent[]
6. 每个事件通过 computeHotspotScore() 计算评分
7. 结果缓存至内存 (LiveCache, 30分钟有效期)
8. 返回给Client -> 渲染3D地球 + 面板
9. Client每60秒自动刷新

3.2 API端点

端点	功能	备注
GET /api/events	环境事件列表	Cache: s-maxage=600, swr=300
GET /api/fires	NASA FIRMS火灾检测	需要 NASA_FIRMS_MAP_KEY
GET /api/location	用户IP地理位置	来源: Vercel Headers
GET /api/health	健康检查	返回 {"status":"ok"}

3.3 数据降级策略

系统设计了三级数据降级机制，确保在任何网络条件下都能正常展示：

Level 1 - Live Data (正常模式)

从NASA EONET v3实时获取数据
成功获取后更新内存缓存 (LiveCache)
缓存有效期: 30分钟

Level 2 - Stale Cache (网络故障第一道防线)

API请求失败时，返回最近一次成功缓存
前端标注数据来源为 "stale-cache"

Level 3 - Bundled Fallback (无缓存时的最后防线)

使用构建时打包的8条样本事件数据
涵盖所有9种事件类别
前端明确标注为样本数据

3.4 数据规范化流程

EONET v3 API同时支持JSON和GeoJSON两种格式。规范化管线处理以下步骤：

1. Zod Schema验证 -> 确保数据结构完整性
2. Category映射 -> EONET分类字符串 -> 内部EventCategory枚举
3. 坐标提取 -> Point直接取值，Polygon使用Shoelace公式计算质心
4. 观测提取 -> 从geometry数组提取历史观测，按时间排序+去重(最多30条)
5. 时间推导 -> 从观测记录推导startedAt(最早)和updatedAt(最新)
6. 评分计算 -> 对每个事件执行computeHotspotScore()
7. 来源解析 -> 提取数据源名称和URL

4 核心算法：热点评分系统

4.1 算法概述

Hotspot Score (热点评分)

是一个完全确定性的0-100评分系统。每一次调用使用相同的输入和引用日期，产生完全相同的输出。算法不使用Math.random()或Date.now()，确保SSR渲染、测试和回放场景下的可重现性。

4.2 评分因子与权重

评分因子	分值	评分细则
1. 类别基础权重	0-30	Wildfire/Volcano(25), Storm(20), Flood(18), Dust/Haze/Drought/Landslide(15), Ice(10), Other(10)
2. 数据时效性	0-25	<6h:25, <12h:22, <24h:18, <48h:12, <7d:5, >7d:2, unknown:0
3. 量级数据	0-15	Value+Unit齐全:15, 仅一项:8, 无:0
4. 事件状态	0-10	Open/Active:10, Closed:0
5. 观测数量	0-10	>=5:10, >=3:7, >=2:5, 1:3
6. FIRMS火点密度	0-10	>100:10, >50:8, >20:6, >10:4, >0:2, 0:0

4.3 算法伪代码

```
function computeHotspotScore(event, referenceDate):
  total = 0
  explanations = []

  // 1. 类别权重
  total += CATEGORY_WEIGHTS[event.category] // 0-30
  explanations.push(categoryLabel)

  // 2. 时效性
  hoursAgo = hoursBetween(referenceDate, event.updatedAt)
  recencyScore = bucketByRecency(hoursAgo) // 0-25
  total += recencyScore

  // 3. 量级
  if event.magnitudeValue AND event.magnitudeUnit:
    total += 15
  elif event.magnitudeValue OR event.magnitudeUnit:
    total += 8
  else: total += 0

  // 4. 状态
  total += event.isOpen ? 10 : 0

  // 5. 观测数量
  total += bucketByObsCount(event.observationCount) // 3-10

  // 6. FIRMS火点密度
  total += bucketByFireDensity(event.nearbyFireCount) // 0-10

  return { score: min(total, 100), explanation }
```


4.4 设计原则

- 完全确定性：所有输入相同时，输出100%一致
- 可解释性：每个分数都附带人类可读的解释列表
- 纯函数：不依赖外部状态、不产生副作用
- 可独立测试：无需mock即可完整测试

5 核心算法：个人相关性评分

5.1 算法概述

Personal Relevance Score (个人相关性评分)

衡量一个环境事件对特定用户的重要程度。通过四个维度加权计算，结果区间0-100。

5.2 评分因子

评分维度	权重	计算方式
1. 距离	45%	<50km:45, <100km:35, <250km:25, <500km:15, <1000km:5, >1000km:0
2. 热点严重性	30%	0.3 x 事件Hotspot Score
3. 数据时效	15%	<6h:15, <12h:12, <24h:9, <48h:5, <72h:2, older:0
4. 时间上下文	10%	白天(06-20)且数据<24h:10, 白天:5, 夜间/未知:0

5.3 算法公式

$PersonalRelevance = Proximity + Severity + Recency + TimeContext$

$Proximity = f(distanceKm) \times 0.45$ (逆距离分桶)

$Severity = 0.3 \times hotspotScore$ (严重性缩放)

$Recency = f(recencyHours) \times 0.15$ (时效分桶)

$TimeContext = f(isDaytime, recency) \times 0.10$ (白天+新鲜)

$Result = clamp(round(raw), 0, 100)$

5.4 应用场景

- Local Signal Bar: 底部栏显示最近的威胁事件及个人匹配度
- Nearest Event: 地球上自动定位到离用户最近的环境事件
- 颜色编码: >70 绿色(高关注), >40 黄色(中等), <40 灰色(低关注)

6 核心算法：Haversine距离计算

6.1 算法原理

使用Haversine公式计算地球表面两点之间的大圆距离。该公式假设地球为球体（半径6371km），对于环境监测场景足够精确。

6.2 数学公式

Haversine Formula:

$$a = \sin^2(\Delta\phi/2) + \cos(\phi_1) * \cos(\phi_2) * \sin^2(\Delta\lambda/2)$$

$$c = 2 * \operatorname{atan2}(\sqrt{a}, \sqrt{1-a})$$

$$d = R * c$$

where:

ϕ = latitude in radians

λ = longitude in radians

R = 6,371 km (Earth mean radius)

d = great-circle distance in km

6.3 实现特点

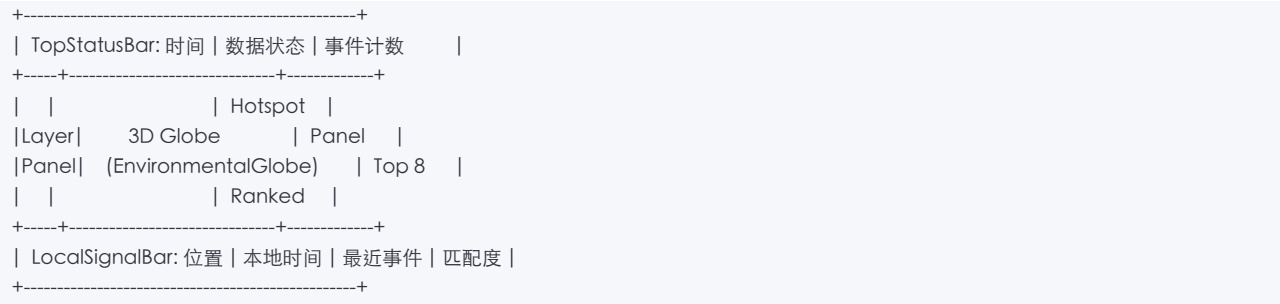
- 纯函数实现，零外部依赖
- 支持 Node.js / Edge / Browser 三种运行时
- 输入校验：经纬度范围检查 $[-90, 90] \times [-180, 180]$
- 短路径优化：同坐标直接返回0，避免浮点精度问题
- 包含完整的JSDoc文档和数学推导注释

6.4 精度说明

Haversine公式在球体近似下的误差约为0.3%，对于全球环境事件定位（通常在10km量级以上）足够精确。如需更高精度（厘米级），可替换为Vincenty公式。但当前场景无需此精度。

7 组件树与UI架构

7.1 页面布局 (桌面端)



7.2 完整组件树

```
RootLayout (app/layout.tsx)
  Home (app/page.tsx) ["use client"]
    EnvironmentalGlobe (dynamic, ssr:false)
      DebugGlobe
      GlobeGL (react-globe.gl)
      HTML Markers (自定义DOM渲染)
      Path Overlay (观测历史路径)
      Camera Controls (zoom/rotate/focus)
    TopStatusBar
    LayerPanel (事件分类过滤)
    HotspotPanel (热点排名 Top 8)
    LocalSignalBar (个人相关性)
    EventDetails (事件详情弹窗)
    ObservationTimeline (观测时间线)
    EpicSatellite (EPIC卫星影像)
    GlobeLegend (图例)
    ThemeToggle (暗色/亮色切换)
    MobileEventSheet (移动端底部抽屉)
```

7.3 数据流 (State Management)

Home组件集中管理所有状态，通过props向子组件传递：

数据状态：

```
events[], eventsLoading, eventsError, dataSource, lastUpdated
```

位置状态：

```
userLocation { city, country, lat, lng, timezone }, hasFirmsKey
```

UI状态：

```
selectedEventId, selectedCategory, refreshIn, showInfo
```

派生数据 (useMemo)：

```
filteredEvents, nearestEvent, nearestDistance, personalRelevance,
localTime, categoryCounts, selectedEvent
```

8 3D地球渲染引擎

8.1 技术选型

使用 `react-globe.gl` (v2.38) 封装 `Three.js` 进行3D地球渲染。`react-globe.gl`是最成熟的React 3D地球库，提供丰富的标记、路径、大气等可视化能力。

关键决策：使用 `htmlElementsData` API (HTML标记) 而非原生`Three.js`标记。HTML标记系统更可靠，支持复杂样式和事件处理。

8.2 渲染配置

属性	值	说明
<code>globeImageUrl</code>	<code>/textures/earth-light.jpg</code>	2K地球纹理 (jsDelivr CDN)
<code>backgroundColor</code>	暗色 <code>#0a1226</code> / 亮色 <code>#d5dbe3</code>	跟随系统主题
<code>atmosphereColor</code>	<code>#5599DD</code> / <code>#3388CC</code>	大气光晕效果
<code>atmosphereAltitude</code>	0.3	大气层高度
<code>htmlElementsData</code>	<code>MarkerData[]</code>	HTML标记数据源

8.3 标记系统 (Marker System)

- 密度控制: 根据缩放级别动态过滤低分事件 (alt>4: minScore=50, alt>3: minScore=30)
- 最多显示500个标记，按`hotspotScore`降序排列
- 用户位置: 蓝色菱形旋转45度，与其他标记视觉区分
- 选中标记: 白色边框 + `scale(1.5)` 放大效果
- 事件标记: 圆形，大小 = $6 + (\text{score}/100) * 8 \text{ px}$
- 颜色: 9种分类色，来自 `CATEGORY_COLORS_HEX`

8.4 相机控制

- Zoom In/Out: `altitude` 范围 `[0.8, 8.0]` 倍率缩放
- Auto-Rotate: 3秒无操作后自动旋转，用户交互时暂停
- Focus Event: 选中事件时相机飞至事件坐标 (800ms动画)
- Observation Jump: 点击观测历史时飞至对应坐标

8.5 可视化特效

- 观测历史路径: 选中事件时显示观测位置变化的虚线路径（最多30个点）
- 路径动画: `dashAnimateTime=8000ms` 流动虚线效果
- 暗色/亮色主题: 地球背景色和大气色随系统主题自动切换

9 数据源与容错机制

9.1 NASA EONET v3

NASA Earth Observatory Natural Event Tracker (EONET) v3 是主要数据源。

API端点: <https://eonet.gsfc.nasa.gov/api/v3/events>

请求参数: `status=open&days=60&limit=500`

数据格式: JSON (首选) / GeoJSON (备用)

超时设置: 15秒

重试策略: 最多2次，指数退避 (500ms, 1500ms)

9.2 事件分类映射

EONET类别	内部枚举	中文
Wildfires	wildfire	野火
Severe Storms	severe-storm	强风暴
Volcanoes	volcano	火山
Floods	flood	洪水
Drought	drought	干旱
Dust and Haze	dust-haze	沙尘/雾霾
Landslides	landslide	山体滑坡
Sea and Lake Ice	sea-lake-ice	海冰/湖冰

9.3 关键词后备匹配

当EONET分类无法精确匹配时，将通过正则表达式对事件标题进行关键词匹配来推断分类。支持 wildfire/fire/storm/cyclone/hurricane/volcano/flood/drought/dust/haze/landslide/sea ice/lake ice 等关键词的模糊匹配。

9.4 几何数据处理

Point类型: 直接取坐标 [lng, lat]

Polygon类型: 计算多边形质心 (坐标算术平均或Shoelace公式)

其他类型 (LineString等): 跳过

筛选规则: 取最新日期的有效几何记录

9.5 NASA FIRMS (可选)

NASA FIRMS (Fire Information for Resource Management System) 提供卫星热异常检测。

API: firms.modaps.eosdis.nasa.gov/api/area/

配置: 需要 NASA_FIRMS_MAP_KEY (免费注册)

未配置时: 前端优雅降级, 显示"Detailed fire detections unavailable"

10 部署与运维

10.1 部署平台

配置项	值
生产URL	planet-pulse-eta.vercel.app
部署平台	Vercel (Hobby Plan)
框架	Next.js 16 App Router
构建命令	next build
分支	main (自动部署)
环境变量	NEXT_PUBLIC_SITE_URL, NASA_FIRMS_MAP_KEY

10.2 已验证路由

路由	状态	说明
/	200 OK	完整 Planet Pulse 应用
/api/health	200 OK	{ "status": "ok" }
/api/events	200 OK	EONET实时数据+降级
/api/location	200 OK	Vercel IP地理定位
/api/fires	200 OK	优雅降级 (无FIRMS密钥)

10.3 缓存策略

- API缓存: Cache-Control: public, s-maxage=600, stale-while-revalidate=300
- 服务端内存缓存: 30分钟有效期 (LiveCache)
- Client自动刷新: 每60秒轮询一次

10.4 可观测性

- 响应头: X-Data-Source (live/stale-cache/fallback)
- 降级原因: X-Degraded-Reason 头标识具体失败原因
- 日志: 服务端console.error记录API异常

11 测试体系

11.1 测试框架

- 框架: Vitest (与Vite生态深度集成)
- 运行: npm test (vitest run)
- 类型检查: npm run typecheck (tsc --noEmit)
- 代码检查: npm run lint (ESLint)

11.2 测试覆盖

测试文件	用例数	覆盖范围
eonet-normalization.test.ts	34	EONET分类映射、几何选择、坐标验证、URL净化
hotspot-score.test.ts	15	热点评分边界、确定性、所有因子
personal-relevance.test.ts	16	个人相关性边界、权重计算
distance.test.ts	13	Haversine距离、坐标校验、边界条件
health.test.ts	N/A	API健康检查端点
event-visuals.test.ts	N/A	可视化工具函数

11.3 测试原则

- 评分算法测试必须覆盖边界条件 (0分、100分、极端输入)
- 所有数据规范化测试覆盖有效输入和无效输入
- 距离测试覆盖同坐标、赤道、极地、日期变更线场景
- 测试中使用固定的referenceDate保证确定性

12 隐私与数据伦理

12.1 隐私设计

- 不存储IP地址
- 不记录任何日志中的位置信息
- 位置数据仅从Vercel Headers服务端派生（城市级精度）
- 无位置追踪Cookie
- 精确浏览器地理定位严格用户主动触发（按钮点击）
- 无位置数据持久化到数据库或存储
- 无用户账户系统
- 无身份认证机制
- 无跟踪/分析代码

12.2 数据来源声明

Planet

Pulse使用卫星和公共环境数据用于态势感知和教育探索。明确声明：不是官方紧急警报服务。不应依赖其进行安全关键决策。

所有数据均来自公开的NASA API，不涉及用户上传或用户生成内容。

12.3 数据局限性透明声明

- EONET事件来自多个国际机构，并非所有事件都能实时捕获
- 部分事件缺少量级或科学测量数据
- 热点评分是相对排名工具，非科学危害评估
- FIRMS检测热异常（热点像素），可能包含非火灾热源
- IP地理定位精度为城市级别
- EONET API可能不可用，降级数据会明确标注

13 未来规划 (Phase 2)

以下为Phase 2规划的功能，按优先级排列：

功能	描述
CesiumJS 集成	替换 react-globe.gl，获得更高保真度地球渲染
NASA GIBS 图层	卫星影像图层叠加（温度、植被、气溶胶等）
事件历史时间线	带播放控件的时间轴回放功能
空气质量数据	集成AQI等空气质量监测数据
事件聚类	远距离视角下的聚合标记
可分享URL	事件深度链接 (URL参数已部分实现)
PWA支持	离线地球浏览
多语言支持	国际化

13.1 架构演进方向

- Globe组件边界已预留，CesiumJS替换只需更换实现层
- 评分系统已模块化，新增数据源只需添加新的API Route + 规范化函数
- 无状态设计便于水平扩展

Thank you!

Planet Pulse - See what Earth is signaling now.
Copyright (c) 2026 John Zhou. Apache License 2.0.